

## **Final Year Project**

ApplicationX: Automated CI/CD Pipeline and Deployment Automation  
Project Plan

**Student:** Shawn Chow Wei Jie (U2320168C)

**Supervisor:** Dr Chng Eng Siong

## Project Overview

ApplicationX is *[a short description of the system: what it does, who uses it, and what it is built with — for example, a web application with a Python backend, a PostgreSQL database, and a speech recognition service developed by the NTU Speech Lab]*. The system has been developed and maintained by successive Final Year Project students and researchers.

While the application itself is functional, its software delivery process remains manual. Builds and deployments are performed by running commands by hand on a specific machine, there is no automated testing, and configuration values and credentials are stored in plaintext files. As a result, releases are slow and error prone, the version currently deployed is difficult to determine, and there is no straightforward way to roll back a faulty release.

## Project Objectives

The objective of this project is to design and implement an automated Continuous Integration and Continuous Deployment (CI/CD) pipeline for ApplicationX, so that changes can be built, verified and deployed in a consistent and repeatable manner without manual intervention.

A previous Final Year Project introduced a CI/CD pipeline for another system in the NTU Speech Lab and demonstrated that this approach works well in this environment. This project applies the same approach to ApplicationX, and extends it in four areas that were not covered previously: an automated test suite enforced as a quality gate before any change can be merged; handling of database schema changes as a controlled step in the deployment process; a containerised development environment that allows a future student to run the entire system with a single command; and a rollback procedure that is verified by deliberately deploying a faulty build.

## Project Scope

### Containerisation and Development Environment

- Write Dockerfiles for each service so that the application builds and runs identically in development and in deployment
- Provide a Docker Compose configuration that starts the entire system, including its database and dependent services, with a single command
- Record the current manual release process as a baseline, so that the improvement can be measured at the end of the project

### Continuous Integration

- Set up a GitHub Actions workflow triggered automatically on every pull request and merge
- Run code linting and formatting checks on every change
- Write an automated test suite covering the critical paths of the application, and enforce it as a quality gate before merging
  - Unit tests for application logic
  - Integration tests against a real database and dependent services
  - A minimum code coverage threshold that must be met before a change can be merged
- Scan the source code, dependencies and container images for known vulnerabilities and exposed secrets, and report the results on the pull request
- Enable automated dependency update proposals
- Build and publish versioned container images to a container registry

## **Continuous Deployment**

- Set up a Kubernetes cluster on laboratory hardware, with an equivalent local cluster for development
- Package each service as a Helm chart, with separate configuration for the development and production environments
- Configure Argo CD to automatically synchronise the cluster with the deployment repository, using Git as the single source of truth
- Store credentials in encrypted form using Sealed Secrets, so that no secret is committed in plaintext
- Run database schema migrations as a controlled step before a new version receives traffic, with a documented rollback path
- Deploy new versions progressively, with automatic rollback if health checks fail
- Verify the rollback procedure by deliberately deploying a faulty build and measuring how quickly it is detected and reverted

## **Monitoring**

- Deploy Prometheus and Grafana, and expose application metrics from the services
- Build dashboards covering request rate, error rate, response time and resource usage
- Configure alerts on defined thresholds

## **Proposed Technology Stack**

- Version control and continuous integration: Git, GitHub, GitHub Actions
- Automated testing: pytest and code coverage reporting
- Containerisation: Docker and Docker Compose
- Container registry: GitHub Container Registry
- Security scanning: Trivy and Dependabot
- Orchestration and packaging: Kubernetes (k3s) and Helm
- Continuous deployment: Argo CD and Argo Rollouts
- Secret management: Sealed Secrets
- Monitoring: Prometheus and Grafana

All of the tools listed above are open source and can be self-hosted, so the pipeline does not depend on a commercial cloud provider.

### Stretch goal: Preview Environments

- Automatically create a temporary environment for each pull request, and remove it when the pull request is closed

### Stretch goal: Backup and Recovery

- Implement and test an automated backup and restore procedure for the database and stored files

## Project Schedule

| Index | Task Name                                       | Start Date  | End Date    |
|-------|-------------------------------------------------|-------------|-------------|
|       | <b>Preparation</b>                              |             |             |
| 1     | Scope Definition and Discussion with Supervisor | 25 Aug 2026 | 14 Sep 2026 |
| 2     | Review of Existing System and Pipeline Design   | 14 Sep 2026 | 12 Oct 2026 |
|       | <b>Implementation</b>                           |             |             |
| 3     | Containerisation and Development Environment    | 12 Oct 2026 | 9 Nov 2026  |
| 4     | Continuous Integration and Automated Testing    | 9 Nov 2026  | 21 Dec 2026 |
| 5     | Kubernetes Setup and Helm Charts                | 21 Dec 2026 | 18 Jan 2027 |
| 6     | Continuous Deployment and Release Strategy      | 18 Jan 2027 | 22 Feb 2027 |
| 7     | Monitoring and Alerting                         | 22 Feb 2027 | 15 Mar 2027 |
| 8     | Evaluation, Buffer Time and Stretch Goals       | 15 Mar 2027 | 19 Apr 2027 |
|       | <b>Report</b>                                   |             |             |
| 9     | Project Plan                                    | 25 Aug 2026 | 31 Aug 2026 |

| <b>Index</b> | <b>Task Name</b> | <b>Start Date</b> | <b>End Date</b> |
|--------------|------------------|-------------------|-----------------|
| 10           | Interim Report   | 1 Sep 2026        | 11 Jan 2027     |
| 11           | Final Report     | 12 Jan 2027       | 19 Apr 2027     |
| 12           | Presentation     | 19 Apr 2027       | 10 May 2027     |

*[Dates to be confirmed against the official FYP calendar.]*

## Gantt Chart

[illegible]